



**BASE DE
DATOS III**

Ferret

Lautaro Rivieri
Mariano Capella
Gabriel Osemberg
Federico Montoro

Contenidos

01 El problema

02 DER

03 Data Seeding

04 SQL Avanzado

05 Estrategia de Indexación

06 Thick Database

07 Arquitectura Híbrida

08 Consolidación Full Stack



¿Qué problema resuelve nuestro sistema?



Gestión centralizada

Unifica productos y clientes en una sola plataforma.



Control de stock

Permite administrar inventario y movimientos entre sucursales.



Ventas automatizadas

Registra ventas y actualiza el stock de forma automática.



Información confiable

Facilita el análisis y la toma de decisiones con datos consistentes.

Diagrama Entidad-Relación

[Mostrar desde Github](#)

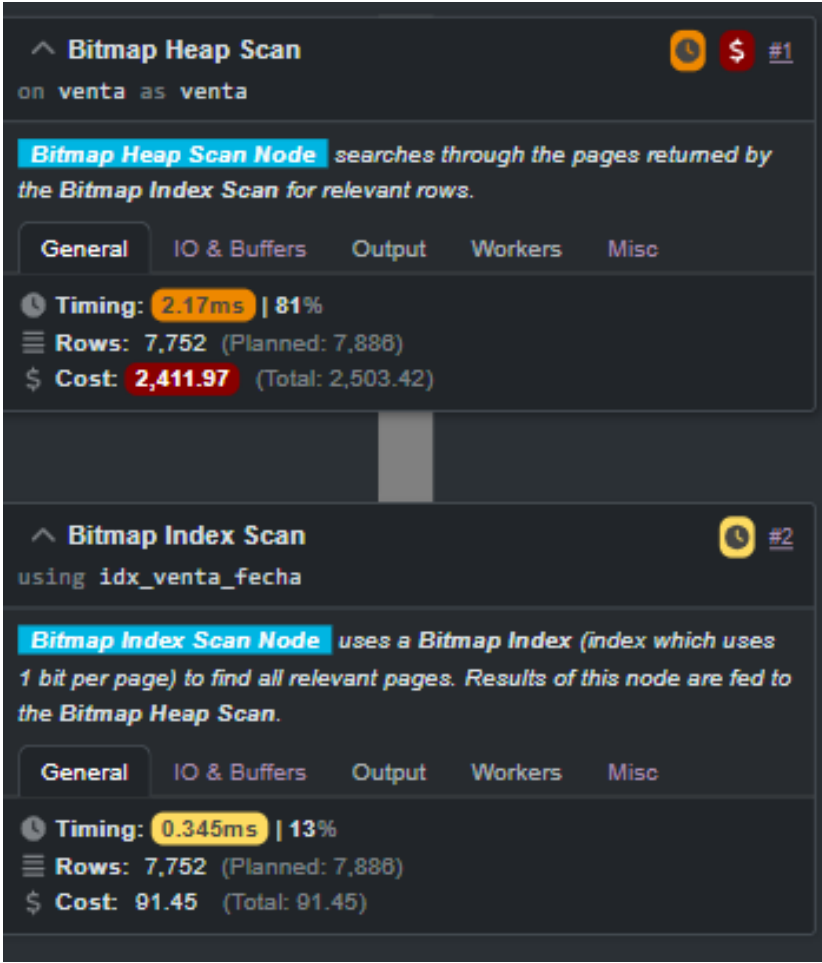


Data Seeding



6	15	5	200	100
sucursal	camion	cargo	empleado	marca
50	30K	80K	480K	100K
categoria	producto	sku	stock	cliente
	200K	600K	150K	
	venta	venta_linea	movimiento_stock	

Estrategia de Indexación

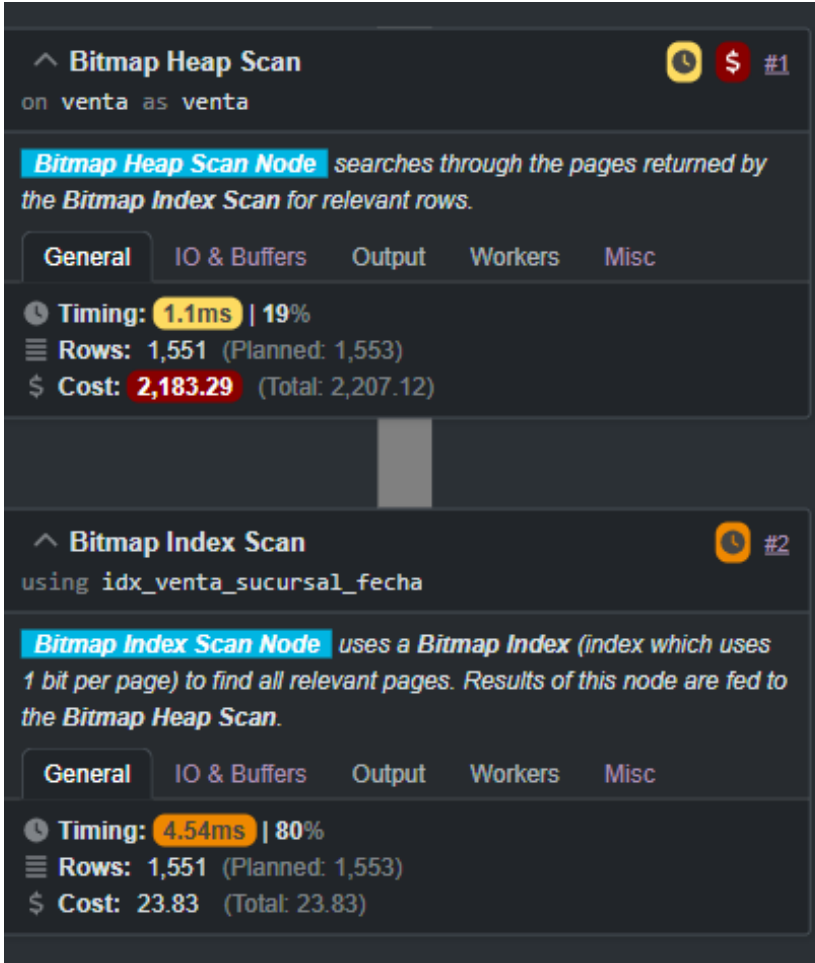


idx_venta_fecha

B-Tree sobre **venta.fecha_venta**

Búsquedas por rango de fechas.

Tiempo: **0.345 ms**.

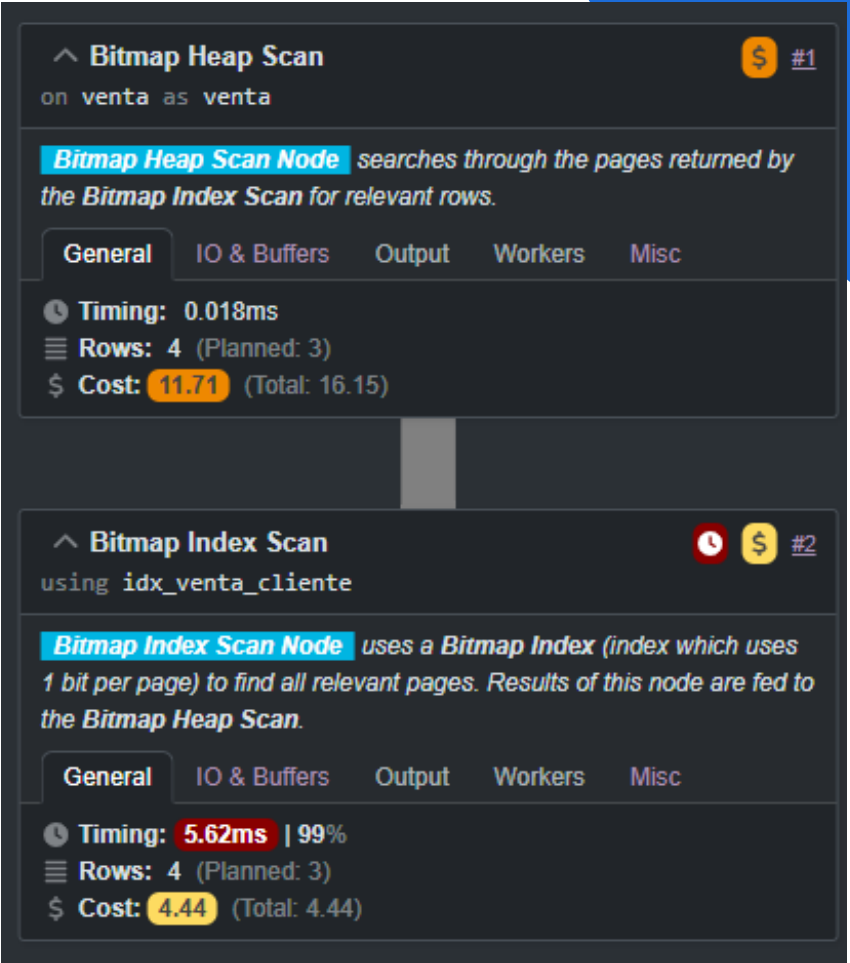


idx_venta_sucursal_fecha

B-Tree compuesto sobre **sucursal_id** y **fecha_venta**

Filtros combinados de sucursal + período.

Tiempo: **4.54 ms**.

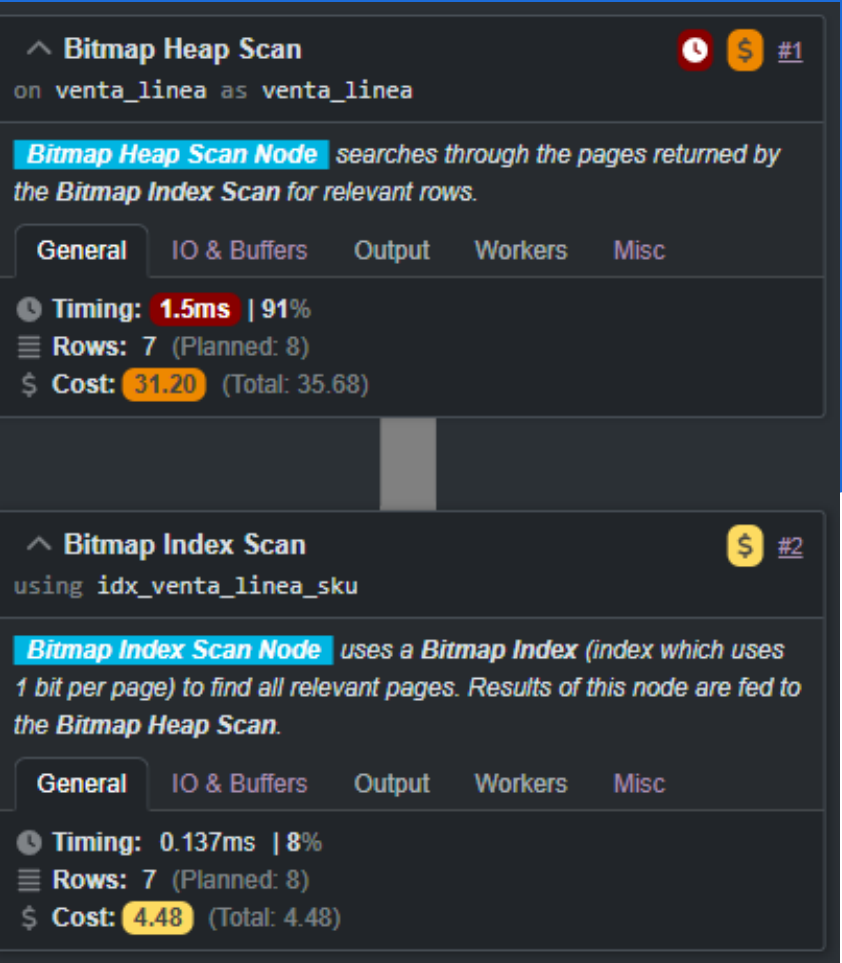


idx_venta_cliente

B-Tree sobre **venta.cliente_id**

Lookup de ventas por cliente.

Tiempo: **5.62 ms**.

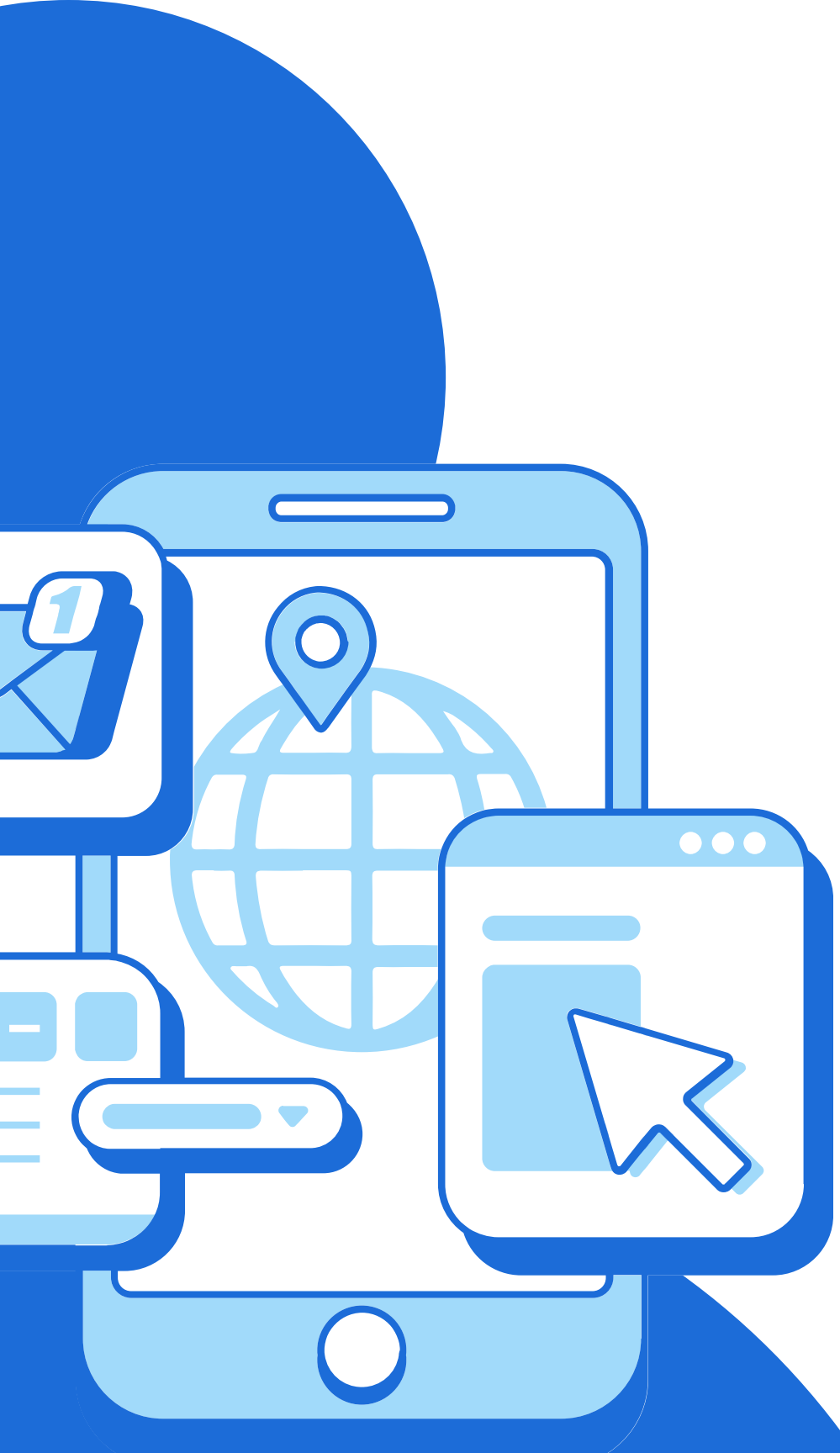


idx_venta_linea_sku

B-Tree sobre **venta_linea.sku_id**

Lookup de líneas de venta por SKU (tabla de 600k filas).

Tiempo: **0.137 ms**.



SQL Avanzado

- **Top 3 vendedores por sucursal (RANK())**
- **Acumulado mensual + promedio móvil 3M (SUM/AVG OVER)**
- **Árbol de categorías (CTE Recursiva)**
3 niveles de profundidad. Ruta completa con indentación visual.
- **Cadena de mando (CTE Recursiva)**
Sube por supervisor_id desde un empleado dado hasta la raíz del organigrama.
- **Bonus: Window + GIN sobre JSONB**
Top 10 productos de material acero por unidades vendidas.
Demuestra que el índice GIN de Stream C se usa en consultas de negocio reales:

[Mostrar desde Github](#)

Thick Database

Lógica y Seguridad



Abstracción y Transacciones



Auditoría



Seguridad



Postgre + Redis

Alta frecuencia de
lectura



Baja frecuencia de
escritura



Tolerancia a la
consistencia
eventual



[Mostrar desde Github](#)

Consolidación Full Stack

Uso de la API REST desde Postman

Uso de KEYS para invalidar caché de forma selectiva

1

POST - `api/productos/:id`

- Crear nuevo producto

2

PUT - `api/productos/:id`

- Modificar nombre

3

DELETE - `api/productos/:d`

- Baja Lógica

4

POST - `api/venta`

- Consumo del procedimiento almacenado `registrar_venta_simple`
- Se tuvo en cuenta posible condición de carrera





Muchas gracias